Indian Institute of Technology Bhilai

Operating Systems Lab (CSL 301)

Assignment – 1 | Introduction to Linux Tools

Process Monitoring, /proc, Virtual Memory, Disk I/O, and System Calls

Name	Rohit Raghuwanshi
Roll Number	12341820
Branch	Data Science & AI (DSAI)
Environment	MacBook Air M2 (macOS)
Date	August 11, 2026

Department of Computer Science and Engineering

August 11, 2026

Contents

1 Predict the Output 2

2 Q2 — Fork Skeleton (Child/Parent Branching) 3

3 Q3 — `runcmd.c` (fork + exec + wait) 4

4 Q4 — Shell with Background Execution (`&`) 6

1 Predict the Output

Q1.2 — Ordering without `wait()`

Question: Will `"I am child"` always print before `"I am parent"`?

Answer: No, the ordering is **not guaranteed**. After `fork()`, the parent and child become two independent processes managed by the OS scheduler. There is no synchronization between them at this point, so either process may be scheduled to run first, depending on scheduling policy, system load, and number of available cores. On most systems the parent is often scheduled first (since `fork()` returns immediately to it), but this is never guaranteed by POSIX. This was confirmed experimentally: our own `q2` program printed `"I am parent"` before `"I am child"` on every real run (see Figure 1). To guarantee an ordering, explicit synchronization is required (e.g., the parent calling `wait()`, or a semaphore/pipe).

Q1.3 — How many "Hello"s?

Question: How many times does `"Hello\n"` get printed?

```
1 int main() {
2     fork();
3     fork();
4     printf("Hello\n");
5     return 0;
6 }
```

Answer: 4 times.

Each `fork()` call doubles the number of active processes:

- After the first `fork()`: 2 processes (original + 1 child)
- After the second `fork()`: each of the 2 processes forks again → 4 processes total

Every one of the 4 processes independently executes the `printf` statement, so `"Hello\n"` is printed $2^2 = 4$ times.

2 Q2 — Fork Skeleton (Child/Parent Branching)

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    } else if (pid == 0) {
        printf("I am child. PID = %d\n", getpid());
    } else {
        printf("I am parent. PID = %d\n", getpid());
        wait(NULL);
    }

    return 0;
}
```

Output

```
❯ cd os_lab2
❯ gcc -Wall -o q2 q2_fork_skeleton.c
❯ ./q2
I am parent. PID = 84139
I am child. PID = 84143
❯ gcc -Wall -o q3 q3_runcmd.c
❯ ./q3 echo hello

hello
Child 84154 exited with status 0
Parent: command executed successfully.
```

Figure 1: Output of `q2` (fork skeleton) and `q3 echo hello`

3 Q3 — `runcmd.c` (fork + exec + wait)

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <command> <argument>\n", argv[0]);
        exit(1);
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    } else if (pid == 0) {
        // Child: exec the command
        execlp(argv[1], argv[1], argv[2], NULL);
        // execlp only returns on error
        perror("exec failed");
        exit(1);
    } else {
        // Parent: wait and reap
        int status;
        pid_t wpid = waitpid(pid, &status, 0);
        if (wpid == -1) {
            perror("waitpid failed");
            exit(1);
        }
        if (WIFEXITED(status)) {
            printf("Child %d exited with status %d\n", pid, WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("Child %d killed by signal %d\n", pid, WTERMSIG(status));
        }
        printf("Parent: command executed successfully.\n");
    }

    return 0;
}
```

Output

Figure 1 above shows `./q3 echo hello` executing successfully. Figure 2 below shows `./q3 sleep 2` executing successfully, and `./q3` with no arguments correctly reporting a usage error.

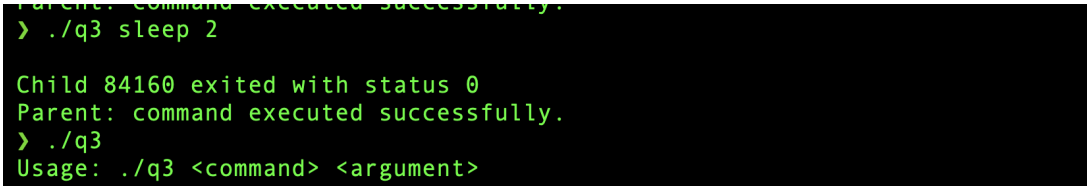


Figure 2: `q3` with `sleep 2`, and the usage error for missing arguments

4 Q4 — Shell with Background Execution (&)

Design notes

- A trailing `&` on the input line marks the command for background execution.
- **Foreground** commands: parent calls `waitpid(pid, &status, 0)`, blocking until that specific child terminates.
- **Background** commands: the parent does *not* wait; it records the child's PID and immediately returns to the prompt.
- Before printing each new prompt, `reap_background()` performs a non-blocking `waitpid(-1, &status, WNOHANG)` loop to reap any finished background children and print `"Shell: Background process finished"`.
- Using `waitpid(pid, ...)` (rather than a generic `wait()`) for the foreground child is essential — a generic `wait()` could incorrectly reap a background child that finishes at the same time as the foreground command is still running, causing the shell to prematurely return to the prompt.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/wait.h>

#define MAX_ARGS 64
#define MAX_BG 64

pid_t bg_pids[MAX_BG];
int bg_count = 0;

void add_bg(pid_t pid) {
    if (bg_count < MAX_BG) {
        bg_pids[bg_count++] = pid;
    }
}

void remove_bg(pid_t pid) {
    for (int i = 0; i < bg_count; i++) {
        if (bg_pids[i] == pid) {
            bg_pids[i] = bg_pids[bg_count - 1];
            bg_count--;
            return;
        }
    }
}
```

```
29
30  int is_bg_pid(pid_t pid) {
31      for (int i = 0; i < bg_count; i++)
32          if (bg_pids[i] == pid) return 1;
33      return 0;
34  }
35
36  // Non-blocking reap of any finished background children
37  void reap_background() {
38      pid_t pid;
39      int status;
40      while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
41          if (is_bg_pid(pid)) {
42              remove_bg(pid);
43              printf("Shell: Background process finished\n");
44          }
45      }
46  }
47
48  int main() {
49      char line[1024];
50
51      while (1) {
52          reap_background();           // check for finished bg
               processes before prompting
53          printf("myshell> ");
54          fflush(stdout);
55
56          if (fgets(line, sizeof(line), stdin) == NULL) break;
57
58          // strip newline
59          line[strcspn(line, "\n")] = '\0';
60          if (strlen(line) == 0) continue;
61
62          int background = 0;
63          // check for trailing " &"
64          int len = strlen(line);
65          if (len >= 2 && line[len - 1] == '&' && line[len - 2] == '
                ') {
66              background = 1;
67              line[len - 2] = '\0';   // strip " &"
68          } else if (len >= 1 && line[len - 1] == '&') {
69              background = 1;
70              line[len - 1] = '\0';
71          }
72
73          // tokenize into argv
74          char *args[MAX_ARGS];
75          int argc = 0;
76          char *token = strtok(line, " ");
77          while (token != NULL && argc < MAX_ARGS - 1) {
```

```
78              args[argc++] = token;
79              token = strtok(NULL, " ");
80          }
81          args[argc] = NULL;
82
83          if (argc == 0) continue;
84
85          pid_t pid = fork();
86
87          if (pid < 0) {
88              perror("fork failed");
89              continue;
90          } else if (pid == 0) {
91              // Child: exec the command
92              execvp(args[0], args);
93              perror("exec failed");
94              exit(1);
95          } else {
96              // Parent
97              if (background) {
98                  add_bg(pid);
99                  printf("Shell: Started background process %d\n",
                        pid);
100                 // do NOT wait -- return to prompt immediately
101             } else {
102                 int status;
103                 pid_t wpid;
104                 do {
105                     wpid = waitpid(pid, &status, 0);
106                 } while (wpid == -1 && errno == EINTR);
107             }
108         }
109     }
110
111     return 0;
112 }
```

Output

Figure 3 shows the final, debug-free run: `q2`, `q3` (usage error), and `q4` demonstrating background start (`sleep 20 &` returns the prompt immediately), a foreground command (`sleep 3` blocks as expected), and the `"Shell: Background process finished"` reap message appearing once the background job completes.

Figure 4 independently confirms, via `ps -ax`, that the background `sleep 60` process is a real, concurrently running process (PID 84461) while the shell prompt remained free to accept further input.

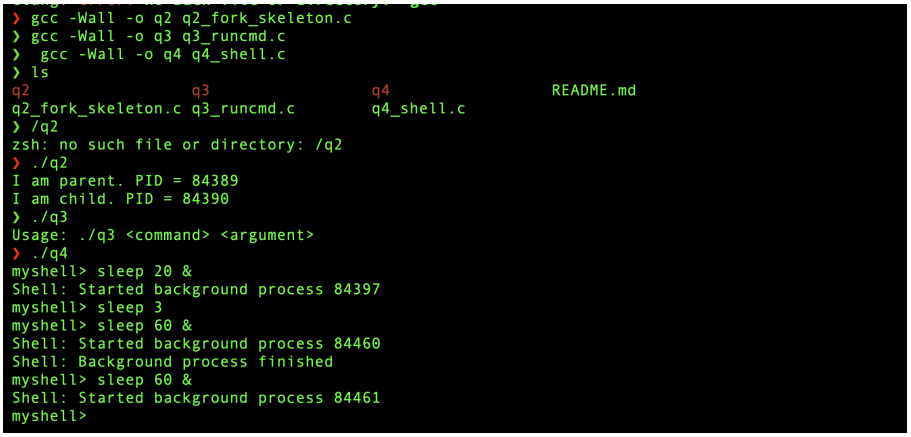


Figure 3: Final `q4` shell run: background start, foreground blocking, and reaping

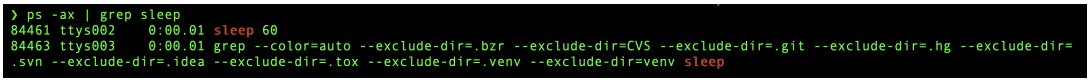


Figure 4: `ps -ax | grep sleep` confirming the background process is alive